

Anexo 03 — Entorno RL de seguimiento (cabeza)

Archivo: Scripts/rl_env.py

Para que sirvio:

Entorno Gymnasium DUMHeadTrackingEnv. Define la observacion (18 dim), la accion (3 joints de la cabeza) y la funcion de recompensa de tracking + foco animatronico + estabilidad, mas el shaping que fuerza el uso del giro de cabeza. Base de las policias v13 y v14c.

```
1 | """DUMHeadTrackingEnv: Gymnasium environment para entrenar una policy
2 | que oriente la cabeza del Pit Droid hacia un target visual.
3 |
4 | Hereda de gymnasium.envs.mujoco.MujocoEnv (v5+, bindings nuevos de mujoco).
5 |
6 | Action space:
7 |     Box(3,) en [-1, 1] -> mapea a ctrlrange de [act_Neck, act_HeadBase, act_HeadRot].
8 |
9 | Observation space:
10 |     Box(13,) = qpos_head(3) + qvel_head(3) + target_dir_local(3) + target_dist(1) + prev_action(3).
11 |
12 | Reward (sin LaTeX):
13 |     r = w_track * exp(-theta^2 / sigma^2)
14 |         - w_smooth * ||a_t - a_{t-1}||^2
15 |         - w_effort * ||a_t||^2
16 |         - w_jitter * ||qvel_head||^2
17 |         + w_alive
18 | """
19 |
20 | from pathlib import Path
21 | import numpy as np
22 | import mujoco
23 | from gymnasium.envs.mujoco import MujocoEnv
24 | from gymnasium.spaces import Box
25 |
26 |
27 | XML_PATH = str(Path(__file__).resolve().parents[1] / "Cuerpo" / "DUM4.xml")
28 |
29 | DEFAULT_REWARD_WEIGHTS = dict(
30 |     w_track=1.0,
31 |     sigma=0.4,
32 |     # rad - ancho del exponencial de tracking.
33 |     # Originalmente 0.1, subido tras smoke test:
34 |     # con 0.1 r_track ≈ 0 a θ>11°, no daba gradient desde poses iniciales.
35 |     # Con 0.4: r_track(7°)=0.97, r_track(28°)=0.29, r_track(45°)=0.08.
36 |     w_smooth=0.15,
37 |     w_effort=0.0075,
38 |     w_alive=0.0,
39 |     w_jitter=0.01,
40 |     # Subido de 0.002: el v8 seguia vibrando al asentar.
41 |     # 0.01 implica ~5x mas penalty por qvel residual.
42 |     w_focus_event=0.0,
43 |     # ELIMINADO (era 5.0): el experto lo descarto, mete escalon discreto
44 |     # en value function y compite con el hold creciente.
45 |     w_focus_hold=0.03,
46 |     # Coeficiente para r_hold = w_hold * sqrt(streak_steps) cuando theta<umbral.
47 |     # sqrt premia mantener prolongado sin explotar (lineal cap saturaria).
48 |     w_post_focus_mult=1.5,
49 |     # Bajado de 2.0: el reward de hold ya empuja "quietud", no hace falta tanto
50 |     # multiplicador.
51 |     # v14b head: guidance shaping para forzar uso de HeadRot en azimuth extremo.
52 |     # Penalty proporcional al mismatch entre HeadRot_qpos y el target_az esperado,
53 |     # S0L0 cuando |target_az| > umbral (no aplica en frontal).
54 |     w_headrot_guidance=0.0,
55 |     # default 0 = sin shaping. v14c uso 2.0 (era 0.5).
56 |     headrot_guidance_az_thr=0.3,
57 |     # rad ~17° - debajo de esto, no se aplica el penalty
58 |     # v14c: penalty DIRECTO a |HeadBase_qpos| cuando target_az es grande.
59 |     # v14b fallo porque la policy saturaba HeadBase a ±0.7 rad (limite fisico) para
60 |     # compensar la falta de HeadRot. Este penalty fuerza explicitamente a NO usar
61 |     # HeadBase para azimuth. HeadBase queda libre para elevation moderada.
62 |     w_headbase_tilt_penalty=0.0,
63 |     # default 0. v14c usa 1.5
64 |     headbase_tilt_az_thr=0.5,
65 |     # rad ~30° - debajo de esto, HeadBase libre
66 |     headbase_tilt_q_thr=0.2,
67 |     # rad ~11° - debajo de esto, HeadBase libre
68 | )
69 |
70 | # Hook de enfoque animatronico
71 | FOCUS_THRESHOLD = 0.07
72 | # rad ≈ 4°, equivale a r_track > 0.6
73 | FOCUS_SETTLING_STEPS = 10
74 | # ignora los primeros N steps para evitar disparo trivial
75 | FOCUS_ANIM_STEPS = 50
76 | # ~1 segundo a 50 Hz de control
77 | # Rangos aleatorios para variar la anim de enfoque entre episodios:
78 | FOCUS_N_CYCLES_RANGE = (1, 4)
79 | # samplea en [low, high) -> 1, 2 o 3 ciclos
80 | FOCUS_AMPLITUDE_RANGE = (0.5, 1.0)
81 | # fracción del lens_max alcanzada en cada ciclo
82 | FOCUS_RESTING_RANGE = (0.0, 0.15)
83 | # offset final como fracción del lens_max
84 |
85 | # Re-trigger del focus (solo si one_shot_focus=False, modo interactivo):
86 | # Si el agente pierde el target sostenidamente, se "rearma" el trigger para que
87 | # vuelva a dispararse cuando recupere el focus.
88 | FOCUS_REARM_LOST_STEPS = 30
89 | # 0.6s a 50Hz fuera de threshold para rearmar
90 | FOCUS_REARM_THRESHOLD_MULT = 3.0
91 | # debe estar a > 3*FOCUS_THRESHOLD para "perdido"
```

```

73 |
74 | # Tarea
75 | TARGET_CONE_AZIMUTH = np.deg2rad(75) # Subido de 60° - exigir mas yaw, forzar a usar HeadRot
76 | TARGET_CONE_ELEVATION = np.deg2rad(45) # Subido de 30° - exigir mas pitch
77 | TARGET_DISTANCE_MIN = 0.3
78 | TARGET_DISTANCE_MAX = 1.0
79 | EPISODE_MAX_STEPS = 500 # 10 s a 50 Hz
80 |
81 | # Movimiento del target en cada episodio (se elige uno random):
82 | TARGET_MODES = ["static", "linear", "circular"]
83 | # Linear: oscila entre dos puntos del cono A<->B con periodo:
84 | TARGET_LINEAR_PERIOD_RANGE = (4.0, 8.0) # segundos
85 | # Circular: orbita alrededor de un punto del cono en un plano perpendicular a la mirada:
86 | TARGET_CIRCULAR_RADIUS_RANGE = (0.05, 0.15) # m
87 | TARGET_CIRCULAR_OMEGA_RANGE = (0.5, 1.5) # rad/s ~ 30-90°/s
88 |
89 | # Identificadores del XML
90 | HEAD_ACTUATORS = ["act_Neck", "act_HeadBase", "act_HeadRot"]
91 | HEAD_JOINTS = ["Neck_joint", "HeadBase_joint", "HeadRotation_joint"]
92 | LENS_ACTUATOR = "act_LenteExt"
93 | LENS_JOINT = "LenteExt_joint"
94 | HEAD_BODY = "LenteExt_link" # body cuyo frame se usa para "donde mira"
95 |
96 | # Eje "forward" de la mirada, en el frame local de HEAD_BODY.
97 | # Esta es una hipotesis inicial - el slide del lente avanza en -Y local,
98 | # por lo tanto el frente del ojo apunta en -Y. Se valida con un test visual.
99 | HEAD_FORWARD_LOCAL = np.array([0.0, -1.0, 0.0])
100 |
101 |
102 | class DUMHeadTrackingEnv(MujocoEnv):
103 |     metadata = {
104 |         "render_modes": ["human", "rgb_array", "depth_array"],
105 |         "render_fps": 50,
106 |     }
107 |
108 |     def __init__(self, reward_weights=None, render_mode=None,
109 |                 one_shot_focus=True, cone_az_deg=None, cone_el_deg=None, **kwargs):
110 |         """one_shot_focus=True (default, modo training): el focus se dispara una sola
111 |         vez por episodio. one_shot_focus=False (modo interactivo): el trigger se rearma.
112 |
113 |         cone_az_deg / cone_el_deg: override del cono de muestreo del target en GRADOS.
114 |         Si None, usa los defaults TARGET_CONE_AZIMUTH/ELEVATION (75°/45°)."""
115 |         self._reward_w = {**DEFAULT_REWARD_WEIGHTS, **(reward_weights or {})}
116 |         self._one_shot_focus = one_shot_focus
117 |         # v14b: cono de muestreo del target (instance-level, overridable)
118 |         self._cone_az = np.deg2rad(cone_az_deg) if cone_az_deg is not None else TARGET_CONE_AZIMUTH
119 |         self._cone_el = np.deg2rad(cone_el_deg) if cone_el_deg is not None else TARGET_CONE_ELEVATION
120 |
121 |         # Obs: qpos_head(3) + qvel_head(3) + qpos_hipbody(1) + qvel_hipbody(1)
122 |         # + target_dir_local(3) + target_dist(1) + target_vel_local(3) + prev_action(3) = 18
123 |         # target_vel_local agregada en v13 para que la policy anticipe el movimiento del target.
124 |         observation_space = Box(low=-np.inf, high=np.inf, shape=(18,), dtype=np.float64)
125 |
126 |         MujocoEnv.__init__(
127 |             self,
128 |             model_path=XML_PATH,
129 |             frame_skip=4,
130 |             observation_space=observation_space,
131 |             render_mode=render_mode,
132 |             **kwargs,
133 |         )
134 |
135 |         # Resolver ids una sola vez
136 |         self._head_act_ids = [
137 |             mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_ACTUATOR, n)
138 |             for n in HEAD_ACTUATORS
139 |         ]
140 |         self._head_joint_ids = [
141 |             mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, n)
142 |             for n in HEAD_JOINTS
143 |         ]
144 |         self._head_qpos_adr = np.array([self.model.jnt_qposadr[j] for j in self._head_joint_ids])
145 |         self._head_dof_adr = np.array([self.model.jnt_dofadr[j] for j in self._head_joint_ids])
146 |
147 |         self._lens_act_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_ACTUATOR, LENS_ACTUATOR)
148 |         self._lens_jnt_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, LENS_JOINT)
149 |         self._lens_max = float(self.model.jnt_range[self._lens_jnt_id, 1])
150 |
151 |         # HipBody_joint para incluir en la observacion: el agente "ve" el torso oscilando
152 |         hb_jid = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, "HipBody_joint")
153 |         self._hipbody_qpos_adr = int(self.model.jnt_qposadr[hb_jid])
154 |         self._hipbody_dof_adr = int(self.model.jnt_dofadr[hb_jid])
155 |
156 |         # Site lens_center: origen geometrico real del rayo de mirada
157 |         # (resuelve "desfase constante" causado por el offset body.xpos -> centro del lente)

```

```

158 |         self._lens_center_site_id = mujoco.mj_name2id(
159 |             self.model, mujoco.mjtObj.mjOBJ_SITE, "lens_center"
160 |         )
161 |
162 |         target_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, "target")
163 |         self._target_mocap_idx = int(self.model.body_mocapid[target_body_id])
164 |         self._head_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, HEAD_BODY)
165 |
166 |         # Override del action_space (MujocoEnv default lo deriva del XML - 14 dims)
167 |         self.action_space = Box(low=-1.0, high=1.0, shape=(3,), dtype=np.float32)
168 |
169 |         self._head_ctrl_lo = np.array(
170 |             [self.model.actuator_ctrlrange[a, 0] for a in self._head_act_ids]
171 |         )
172 |         self._head_ctrl_hi = np.array(
173 |             [self.model.actuator_ctrlrange[a, 1] for a in self._head_act_ids]
174 |         )
175 |
176 |         # Estado interno (se reinicia en reset_model)
177 |         self._prev_action = np.zeros(3, dtype=np.float32)
178 |         self._step_count = 0
179 |         self._focus_triggered = False
180 |         self._focus_anim_step = -1
181 |         self._focus_hold_streak = 0 # steps consecutivos con theta < umbral
182 |         self._steps_out_of_focus = 0 # solo se usa cuando one_shot_focus=False (re-arm)
183 |         self._target_world = np.zeros(3)
184 |         self._target_world_prev = np.zeros(3) # para diferencia finita -> target_vel_local
185 |         # Parametros aleatorios de la anim de enfoque (resamplear en reset)
186 |         self._focus_n_cycles = 2
187 |         self._focus_amplitude = 1.0
188 |         self._focus_resting = 0.0
189 |         # Parametros del movimiento del target (resamplear en reset)
190 |         self._target_mode = "static"
191 |         self._target_a = np.zeros(3)
192 |         self._target_b = np.zeros(3)
193 |         self._target_period = 5.0
194 |         self._target_radius = 0.1
195 |         self._target_omega = 1.0
196 |         self._target_phase = 0.0
197 |         self._target_center_for_orbit = np.zeros(3)
198 |         self._head_pos_init = np.zeros(3)
199 |
200 |         # ----- Reset / sampleo del target -----
201 |
202 |         def reset_model(self):
203 |             self.set_state(self.init_qpos.copy(), self.init_qvel.copy())
204 |             mujoco.mj_forward(self.model, self.data)
205 |
206 |             # Origen del target = posicion world del head body en pose neutra
207 |             head_pos = self.data.xpos[self._head_body_id].copy()
208 |             head_mat = self.data.xmat[self._head_body_id].reshape(3, 3).copy()
209 |             forward_world_init = head_mat @ HEAD_FORWARD_LOCAL
210 |             self._head_pos_init = head_pos.copy()
211 |             self._forward_world_init = forward_world_init.copy()
212 |
213 |             # ---- Sampleo del modo de movimiento ----
214 |             mode_idx = int(self.np_random.integers(0, len(TARGET_MODES)))
215 |             self._target_mode = TARGET_MODES[mode_idx]
216 |
217 |             # Sample primer punto (siempre); para "static" es el unico
218 |             self._target_a = self._sample_point_in_cone()
219 |
220 |             if self._target_mode == "linear":
221 |                 self._target_b = self._sample_point_in_cone()
222 |                 self._target_period = float(self.np_random.uniform(*TARGET_LINEAR_PERIOD_RANGE))
223 |             elif self._target_mode == "circular":
224 |                 self._target_radius = float(self.np_random.uniform(*TARGET_CIRCULAR_RADIUS_RANGE))
225 |                 self._target_omega = float(self.np_random.uniform(*TARGET_CIRCULAR_OMEGA_RANGE))
226 |                 self._target_phase = float(self.np_random.uniform(0.0, 2.0 * np.pi))
227 |                 self._target_center_for_orbit = self._target_a.copy()
228 |
229 |             # Setear target inicial
230 |             self._update_target_position(sim_time=0.0)
231 |             self.data.mocap_pos[self._target_mocap_idx] = self._target_world
232 |
233 |             # Settling con ctrl=0 para que la fisica se asiente
234 |             self.data.ctrl[:] = 0.0
235 |             for _ in range(20):
236 |                 mujoco.mj_step(self.model, self.data)
237 |
238 |             # Estado interno
239 |             self._prev_action = np.zeros(3, dtype=np.float32)
240 |             self._step_count = 0
241 |             self._focus_triggered = False
242 |             self._focus_anim_step = -1

```

```

243 |         self._focus_hold_streak = 0
244 |         self._target_world_prev = self._target_world.copy()
245 |
246 |         # Aleatorizar la anim de enfoque para variedad visual
247 |         self._focus_n_cycles = int(self.np_random.integers(*FOCUS_N_CYCLES_RANGE))
248 |         self._focus_amplitude = float(self.np_random.uniform(*FOCUS_AMPLITUDE_RANGE))
249 |         self._focus_resting = float(self.np_random.uniform(*FOCUS_RESTING_RANGE)) * self._lens_max
250 |
251 |         return self._get_obs()
252 |
253 |     def set_cone_limits(self, az_rad: float, el_rad: float):
254 |         """v14_head: callback puede actualizar los limites del cono per-step.
255 |         Si nunca se llama, se usan los defaults (TARGET_CONE_*)."""
256 |         self._cone_az = float(az_rad)
257 |         self._cone_el = float(el_rad)
258 |
259 |     def _sample_point_in_cone(self):
260 |         """Muestrea un punto en el cono frontal a partir de head_pos_init."""
261 |         # v14_head: usar limites instance-level si se setearon, sino los globales
262 |         az_lim = getattr(self, "_cone_az", TARGET_CONE_AZIMUTH)
263 |         el_lim = getattr(self, "_cone_el", TARGET_CONE_ELEVATION)
264 |         az = self.np_random.uniform(-az_lim, az_lim)
265 |         el = self.np_random.uniform(-el_lim, el_lim)
266 |         dist = self.np_random.uniform(TARGET_DISTANCE_MIN, TARGET_DISTANCE_MAX)
267 |         target_dir = self._rot_z(az) @ self._rot_x(el) @ self._forward_world_init
268 |         target_dir /= np.linalg.norm(target_dir) + 1e-9
269 |         return self._head_pos_init + dist * target_dir
270 |
271 |     def _update_target_position(self, sim_time):
272 |         """Actualiza self._target_world segun el modo y el tiempo simulado."""
273 |         if self._target_mode == "static":
274 |             self._target_world = self._target_a
275 |         elif self._target_mode == "linear":
276 |             # phase oscila 0..1.0..1.0 con periodo T (coseno)
277 |             phase = 0.5 * (1.0 - np.cos(2.0 * np.pi * sim_time / self._target_period))
278 |             self._target_world = self._target_a + phase * (self._target_b - self._target_a)
279 |         elif self._target_mode == "circular":
280 |             # Orbita en un plano perpendicular a (target_a - head_pos)
281 |             radial = self._target_center_for_orbit - self._head_pos_init
282 |             r_norm = np.linalg.norm(radial)
283 |             if r_norm < 1e-6:
284 |                 radial = np.array([0.0, -1.0, 0.0])
285 |                 r_norm = 1.0
286 |             radial = radial / r_norm
287 |             up = np.array([0.0, 0.0, 1.0])
288 |             u = np.cross(radial, up)
289 |             if np.linalg.norm(u) < 1e-6:
290 |                 u = np.cross(radial, np.array([1.0, 0.0, 0.0]))
291 |             u = u / (np.linalg.norm(u) + 1e-9)
292 |             v = np.cross(radial, u)
293 |             v = v / (np.linalg.norm(v) + 1e-9)
294 |             ang = self._target_phase + self._target_omega * sim_time
295 |             offset = self._target_radius * (np.cos(ang) * u + np.sin(ang) * v)
296 |             self._target_world = self._target_center_for_orbit + offset
297 |
298 |     @staticmethod
299 |     def _rot_x(a):
300 |         c, s = np.cos(a), np.sin(a)
301 |         return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])
302 |
303 |     @staticmethod
304 |     def _rot_z(a):
305 |         c, s = np.cos(a), np.sin(a)
306 |         return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])
307 |
308 |     # ----- Geometria de la mirada -----
309 |
310 |     def _head_forward_world(self):
311 |         head_mat = self.data.xmat[self._head_body_id].reshape(3, 3)
312 |         return head_mat @ HEAD_FORWARD_LOCAL
313 |
314 |     def _lens_origin_world(self):
315 |         """Origen geometrico real del rayo de mirada (centro visual del lente)."""
316 |         return self.data.site_xpos[self._lens_center_site_id]
317 |
318 |     def _target_local(self):
319 |         """Vector cabeza->target en frame local del lente. Origen: centro visual del lente."""
320 |         origin = self._lens_origin_world()
321 |         head_mat = self.data.xmat[self._head_body_id].reshape(3, 3)
322 |         return head_mat.T @ (self._target_world - origin)
323 |
324 |     def _target_vel_local(self):
325 |         """Velocidad del target en frame local del lente (diferencia finita)."""
326 |         dt = self.frame_skip * float(self.model.opt.timestep)
327 |         head_mat = self.data.xmat[self._head_body_id].reshape(3, 3)

```

```

328 |         vel_world = (self._target_world - self._target_world_prev) / dt
329 |         return head_mat.T @ vel_world
330 |
331 |     def _angle_to_target(self):
332 |         forward = self._head_forward_world()
333 |         delta = self._target_world - self._lens_origin_world()
334 |         delta /= np.linalg.norm(delta) + 1e-9
335 |         return float(np.arccos(np.clip(np.dot(forward, delta), -1.0, 1.0)))
336 |
337 |     # ----- Observacion -----
338 |
339 |     def _get_obs(self):
340 |         qpos_head = self.data.qpos[self._head_qpos_adr]
341 |         qvel_head = self.data.qvel[self._head_dof_adr]
342 |         qpos_hb = float(self.data.qpos[self._hipbody_qpos_adr])
343 |         qvel_hb = float(self.data.qvel[self._hipbody_dof_adr])
344 |         tloc = self._target_local()
345 |         tdist = float(np.linalg.norm(tloc))
346 |         tdir = tloc / (tdist + 1e-9)
347 |         tvel = self._target_vel_local() # nuevo en v13: anticipacion del movimiento
348 |         return np.concatenate(
349 |             [qpos_head, qvel_head, [qpos_hb, qvel_hb], tdir, [tdist], tvel,
350 |              self._prev_action.astype(np.float64)]
351 |         )
352 |
353 |     # ----- Mapeo de accion a ctrl -----
354 |
355 |     def _action_to_head_ctrl(self, action):
356 |         a = np.clip(action, -1.0, 1.0).astype(np.float64)
357 |         # [-1,1] -> [ctrl_lo, ctrl_hi]
358 |         return (a + 1.0) * 0.5 * (self._head_ctrl_hi - self._head_ctrl_lo) + self._head_ctrl_lo
359 |
360 |     def _lens_focus_ctrl(self):
361 |         """Anim de enfoque randomizada por episodio:
362 |         - N ciclos (1, 2 o 3) entre 0 y amplitud*lens_max
363 |         - Tras la anim, queda en resting offset (cerca de 0, leve aleatoriedad)
364 |         """
365 |         if not self._focus_triggered:
366 |             return 0.0
367 |         e = self._focus_anim_step
368 |         if e < 0:
369 |             return 0.0
370 |         if e >= FOCUS_ANIM_STEPS:
371 |             return self._focus_resting
372 |         phase = e / FOCUS_ANIM_STEPS
373 |         peak = self._focus_amplitude * self._lens_max
374 |         return float(peak * 0.5 * (1 - np.cos(2 * self._focus_n_cycles * np.pi * phase)))
375 |
376 |     # ----- step -----
377 |
378 |     def step(self, action):
379 |         # Guardar posicion del target previa para calcular target_vel_local
380 |         self._target_world_prev = self._target_world.copy()
381 |
382 |         # Actualizar posicion del target segun modo y tiempo (movimiento del punto rojo)
383 |         sim_time = self._step_count * self.frame_skip * float(self.model.opt.timestep)
384 |         self._update_target_position(sim_time=sim_time)
385 |         self.data.mocap_pos[self._target_mocap_idx] = self._target_world
386 |
387 |         # Trigger del hook de enfoque (mide antes del paso)
388 |         theta_pre = self._angle_to_target()
389 |         focus_just_triggered = False
390 |
391 |         # Re-arm: si one_shot_focus=False (modo interactivo) y el agente perdio el target
392 |         # sostenidamente, resetear el flag para que pueda volver a dispararse.
393 |         if not self._one_shot_focus and self._focus_triggered:
394 |             if theta_pre > FOCUS_THRESHOLD * FOCUS_REARM_THRESHOLD_MULT:
395 |                 self._steps_out_of_focus += 1
396 |                 if self._steps_out_of_focus >= FOCUS_REARM_LOST_STEPS:
397 |                     self._focus_triggered = False
398 |                     self._focus_anim_step = -1 # tambien resetear anim por las dudas
399 |                     self._steps_out_of_focus = 0
400 |                     # Resamplear parametros aleatorios de la anim para variedad
401 |                     self._focus_n_cycles = int(self.np_random.integers(*FOCUS_N_CYCLES_RANGE))
402 |                     self._focus_amplitude = float(self.np_random.uniform(*FOCUS_AMPLITUDE_RANGE))
403 |                     self._focus_resting = float(self.np_random.uniform(*FOCUS_RESTING_RANGE)) * self._lens_max
404 |             else:
405 |                 self._steps_out_of_focus = 0
406 |
407 |         if (
408 |             not self._focus_triggered
409 |             and self._step_count >= FOCUS_SETTLING_STEPS
410 |             and theta_pre < FOCUS_THRESHOLD
411 |         ):
412 |             self._focus_triggered = True

```

```

413 |         self._focus_anim_step = 0
414 |         focus_just_triggered = True
415 |
416 |     # Construir el vector ctrl completo
417 |     full_ctrl = np.zeros(self.model.nu, dtype=np.float64)
418 |     head_ctrl = self._action_to_head_ctrl(action)
419 |     for i, aid in enumerate(self._head_act_ids):
420 |         full_ctrl[aid] = head_ctrl[i]
421 |     full_ctrl[self._lens_act_id] = self._lens_focus_ctrl()
422 |
423 |     # Simular frame_skip pasos
424 |     self.do_simulation(full_ctrl, self.frame_skip)
425 |
426 |     # Avanzar animacion de enfoque
427 |     if self._focus_anim_step >= 0:
428 |         self._focus_anim_step += 1
429 |         if self._focus_anim_step >= FOCUS_ANIM_STEPS:
430 |             self._focus_anim_step = -1
431 |
432 |     # Reward
433 |     theta_post = self._angle_to_target()
434 |     w = self._reward_w
435 |
436 |     # Actualizar streak de hold: incrementa mientras este en target, reset cuando sale.
437 |     in_target_now = theta_post < FOCUS_THRESHOLD
438 |     if in_target_now:
439 |         self._focus_hold_streak += 1
440 |     else:
441 |         self._focus_hold_streak = 0
442 |
443 |     # Multiplicador para smooth/jitter cuando ya esta en focus (premia quedarse quieto)
444 |     focus_mult = w["w_post_focus_mult"] if self._focus_triggered else 1.0
445 |
446 |     r_track = w["w_track"] * float(np.exp(-(theta_post ** 2) / (w["sigma"] ** 2)))
447 |     r_smooth = -w["w_smooth"] * focus_mult * float(np.sum((action - self._prev_action) ** 2))
448 |     r_effort = -w["w_effort"] * float(np.sum(action ** 2))
449 |     qvel_head = self.data.qvel[self._head_dof_adr]
450 |     r_jitter = -w["w_jitter"] * focus_mult * float(np.sum(qvel_head ** 2))
451 |     r_alive = w["w_alive"]
452 |
453 |     # focus_event: w=0 por default (eliminado por el experto). Lo dejo por compat.
454 |     r_focus_event = w["w_focus_event"] if focus_just_triggered else 0.0
455 |     # focus_hold creciente con sqrt(streak): premia mantener prolongado sin explotar.
456 |     r_focus_hold = w["w_focus_hold"] * np.sqrt(self._focus_hold_streak) if in_target_now else 0.0
457 |
458 |     # v14b: HeadRot guidance – penalty si HeadRot_qpos no apunta al target_az
459 |     # SOLO cuando el target esta lejos (azimuth (>umbral) y el HeadRot esta
460 |     # lejos del valor esperado. Empuja al policy a usar HeadRot en vez de
461 |     # compensar con HeadBase tilt.
462 |     r_headrot_guidance = 0.0
463 |     r_headbase_tilt_penalty = 0.0
464 |     if w.get("w_headrot_guidance", 0.0) > 0.0 or w.get("w_headbase_tilt_penalty", 0.0) > 0.0:
465 |         # Target azimuth relativo al forward del robot (-Y world)
466 |         tgt_dx = float(self._target_world[0] - self._head_pos_init[0])
467 |         tgt_dy_forward = -float(self._target_world[1] - self._head_pos_init[1])
468 |         target_az = float(np.arctan2(tgt_dx, tgt_dy_forward))
469 |         abs_target_az = abs(target_az)
470 |         # HeadRot guidance
471 |         if w.get("w_headrot_guidance", 0.0) > 0.0:
472 |             az_thr = w.get("headrot_guidance_az_thr", 0.3)
473 |             if abs_target_az > az_thr:
474 |                 head_rot_q = float(self.data.qpos[self._head_qpos_adr[2]])
475 |                 diff = head_rot_q - target_az
476 |                 diff = float(np.arctan2(np.sin(diff), np.cos(diff)))
477 |                 r_headrot_guidance = -w["w_headrot_guidance"] * abs(diff)
478 |         # v14c: HeadBase tilt penalty – penalty directo al uso de HeadBase
479 |         # cuando el target requiere yaw (azimuth grande). HeadBase libre
480 |         # cuando target_az es chico (entonces puede pitch para elevation).
481 |         if w.get("w_headbase_tilt_penalty", 0.0) > 0.0:
482 |             az_thr_b = w.get("headbase_tilt_az_thr", 0.5)
483 |             q_thr = w.get("headbase_tilt_q_thr", 0.2)
484 |             if abs_target_az > az_thr_b:
485 |                 head_base_q = float(self.data.qpos[self._head_qpos_adr[1]])
486 |                 excess = max(0.0, abs(head_base_q) - q_thr)
487 |                 r_headbase_tilt_penalty = -w["w_headbase_tilt_penalty"] * excess
488 |
489 |     reward = (r_track + r_smooth + r_effort + r_jitter + r_alive
490 |              + r_focus_event + r_focus_hold + r_headrot_guidance
491 |              + r_headbase_tilt_penalty)
492 |
493 |     self._prev_action = action.astype(np.float32).copy()
494 |     self._step_count += 1
495 |
496 |     obs = self._get_obs()
497 |     terminated = False

```

```
498 | truncated = self._step_count >= EPISODE_MAX_STEPS
499 |
500 | # focus_triggered es el flag interno one-shot (para el efecto del lente).
501 | # Para el indicador VISUAL queremos algo dinamico: "currently in target".
502 | currently_in_target = in_target_now and self._focus_triggered
503 | info = {
504 |     "theta_rad": theta_post,
505 |     "theta_deg": float(np.rad2deg(theta_post)),
506 |     "r_track": r_track,
507 |     "r_smooth": r_smooth,
508 |     "r_effort": r_effort,
509 |     "r_jitter": r_jitter,
510 |     "r_focus_event": r_focus_event,
511 |     "r_focus_hold": r_focus_hold,
512 |     "r_headrot_guidance": r_headrot_guidance,
513 |     "r_headbase_tilt_penalty": r_headbase_tilt_penalty,
514 |     "focus_triggered": currently_in_target, # estado VISUAL dinamico
515 |     "focus_ever_triggered": self._focus_triggered, # one-shot interno
516 |     "focus_anim_step": self._focus_anim_step,
517 |     "focus_hold_streak": self._focus_hold_streak,
518 |     "target_mode": self._target_mode,
519 | }
520 | return obs, reward, terminated, truncated, info
521 |
```